

- app
  - components
    - Navbar.tsx
    - VirtualGuide.tsx
  - planner
    - page.tsx

```

app/components/Navbar.tsx
@@ -109,6 +109,17 @@ export default function Navbar() {
109 109     })
110 110     </Link>
111 111
112 +     <Link
113 +       href="/planner"
114 +       className={`pb-1 hover:text-[#ffcc00] transition-all border-b-2 ${
115 +         pathname === '/planner'
116 +           ? (isDarkMode ? 'text-white border-[#ffcc00]' : 'text-[#1a1a1a] border-[#ffcc00]')
117 +           : 'border-transparent'
118 +       }`}
119 +     >
120 +       Alat Travel
121 +     </Link>
122 +
112 123     <Link
113 124       href="/kontak"
114 125       className={`pb-1 hover:text-[#ffcc00] transition-all border-b-2 ${

```

```

app/components/VirtualGuide.tsx
... @@ -0,0 +1,249 @@
1 + "use client";
2 +
3 + import React, { useState, useEffect, useRef } from 'react';

```

- app/api/auth
  - login
    - route.ts
  - register
    - route.ts

```

app/api/auth/login/route.ts
@@ -10,7 +10,7 @@ export async function POST(request: Request) {
10 10     return NextResponse.json({ message: 'Username dan password wajib diisi!' }, { status: 400 });
11 11   }
12 12
13 -   const user = await prisma.users.findUnique({
13 +   const user = await prisma.user.findUnique({
14 14     where: { username },
15 15   });
16 16

```

```

app/api/auth/register/route.ts
@@ -10,7 +10,7 @@ export async function POST(request: Request) {
10 10   }
11 11
12 12   // Cek apakah username sudah ada
13 -   const existingUser = await prisma.users.findUnique({
13 +   const existingUser = await prisma.user.findUnique({
14 14     where: { username },
15 15   });
16 16
@@ -20,7 +20,7 @@ export async function POST(request: Request) {
20 20
21 21   // Simpan user baru (Role default: admin)
22 22   // TODO: Hash password dengan bcrypt sebelum menyimpan

```

- app
  - components
    - PackingList.tsx
  - planner
    - page.tsx

```

app/components/PackingList.tsx
@@ -2,7 +2,7 @@
2 2
3 3   import React, { useState, useEffect } from 'react';
4 4   import { useTheme } from '@app/context/ThemeContext';
5 - import { Checklist, ShoppingBag, Plus, Trash2, RotateCcw, Check, Sparkles } from 'lucide-react';
6 + import { ShoppingBag, Plus, Trash2, RotateCcw, Check, Sparkles } from 'lucide-react';
7
8 8
9 9
10 10

```

```

app/planner/page.tsx
@@ -4,7 +4,7 @@ import React, { useState, useEffect } from 'react';
4 4   import Navbar from '@app/components/Navbar';
5 5   import Footer from '@app/components/Footer';
6 6   import { useTheme } from '@app/context/ThemeContext';
7 - import { Calculator, Checklist, HelpCircle, Sparkles, MessageSquare, Compass } from 'lucide-react';
8 + import { Calculator, HelpCircle, Sparkles, MessageSquare, Compass } from 'lucide-react';
9 9   import CostEstimator from '@app/components/CostEstimator';
10 10  import PackingList from '@app/components/PackingList';
11 11  import DestinasiQuiz from '@app/components/DestinasiQuiz';
12 12

```

- app/components
  - Breadcrumbs.tsx

```

app/components/Breadcrumbs.tsx
@@ -0,0 +1,26 @@
1 + "use client";
2 +
3 + import React from 'react';
4 + import Link from 'next/link';
5 + import { ChevronRight, Home } from 'lucide-react';
6 + import { usePathname } from 'next/navigation';
7 +
8 + const Breadcrumbs = () => {
9 +   const pathname = usePathname();
10 +   const pathSegments = pathname.split('/').filter((segment) => segment !== '');
11 +
12 +   return (
13 +     <nav className="flex items-center space-y-2 py-4" aria-label="Breadcrumb">
14 +       <ol className="inline-flex items-center space-x-1 md:space-x-3">
15 +         <li className="inline-flex items-center">
16 +           <Link href="/" className="inline-flex items-center text-sm font-medium text-gray-700 hover:text-[#fccc00] dark:text-gray-400 dark:hover:text-white">
17 +             <Home className="w-4 h-4 mr-2" />
18 +             Home
19 +           </Link>
20 +         </li>
21 +       </ol>
22 +     </nav>
23 +   );
24 +

```

app/components  
Breadcrumbs.tsx

```
app/components/Breadcrumbs.tsx
@@ -10,14 +10,34 @@ const Breadcrumbs = () => {
10 10   const pathSegments = pathname.split('/').filter((segment) => segment !== '');
11 11
12 12   return (
13 -     <nav className="flex items-center space-y-2 py-4" aria-label="Breadcrumb">
13 +     <nav className="flex items-center space-x-2 py-4 px-4 md:px-8 max-w-7xl mx-auto w-full z-20 relative" aria-label="Breadcrumb">
14 14     <ol className="inline-flex items-center space-x-1 md:space-x-3">
15 15       <li className="inline-flex items-center">
16 -         <Link href="/" className="inline-flex items-center text-sm font-medium text-gray-700 hover:text-[#ffcc00] dark:text-gray-400
16 +         <Link href="/" className="inline-flex items-center text-xs font-bold uppercase tracking-wider text-gray-500 hover:text-[#ffcc00]
17 -         dark:hover:text-white">
17 +         <Home className="w-4 h-4 mr-2" />
16 +         <Link href="/" className="inline-flex items-center text-xs font-bold uppercase tracking-wider text-gray-500 hover:text-[#ffcc00]
17 +         transition-colors">
17 +         <Home className="w-3.5 h-3.5 mr-2" />
18 18         Home
19 19       </Link>
20 20     </li>
21 +     {pathSegments.map((segment, index) => {
22 +       const href = `/${pathSegments.slice(0, index + 1).join('/')}`;
23 +       const isLast = index === pathSegments.length - 1;
24 +       const label = segment.charAt(0).toUpperCase() + segment.slice(1);
25 +
26 +       return (
27 +         <li key={href} className="inline-flex items-center">
28 +           <ChevronRight className="w-4 h-4 text-gray-400 mx-1" />
29 +           {isLast ? (
```

app/destinasi  
page.tsx

```
app/destinasi/page.tsx
@@ -2,6 +2,7 @@
2 2
3 3   import React, { useState, useEffect } from 'react';
4 4   import Navbar from '@app/components/Navbar';
5 +   import Breadcrumbs from '@app/components/Breadcrumbs';
6 6   import DetailModal from '@app/components/DetailModal';
7 7   import CompareModal from '@app/components/CompareModal';
8 8   import Footer from '@app/components/Footer';
```

app/destinasi  
page.tsx

```
app/destinasi/page.tsx
@@ -237,8 +237,9 @@ export default function DestinasiPage() {
237 237
238 238   /* 3. HAPUS PROPS PROPS LAMA PADA NAVBAR KARENA NAVBAR SUDAH OTOMATIS BACA GLOBAL */
239 239   <Navbar />
240 +   <Breadcrumbs />
241 -   <div className="max-w-7xl mx-auto px-6 md:px-16 pt-40 pb-24 relative z-10">
242 +   <div className="max-w-7xl mx-auto px-6 md:px-16 pt-10 pb-24 relative z-10">
243
244   /* --- WELCOME HERO BANNER & SEARCH BAR --- */
245   <div className={`flex flex-col lg:flex-row lg:items-center justify-between gap-10 mb-14 border-b pb-12 text-left ${
```

▼ app/components  
+ Newsletter.tsx

▼ app/components/Newsletter.tsx

```
... @@ -0,0 +1,85 @@
1 + "use client";
2 +
3 + import React, { useState } from 'react';
4 + import { Mail, Send, Sparkles } from 'lucide-react';
5 + import Swal from 'sweetalert2';
6 + import { useTheme } from '@app/context/ThemeContext';
7 +
8 + const Newsletter = () => {
9 +   const { isDarkMode } = useTheme();
10 +   const [email, setEmail] = useState('');
11 +
12 +   const handleSubmit = (e: React.FormEvent) => {
13 +     e.preventDefault();
14 +     if (!email) return;
15 +
16 +     Swal.fire({
17 +       title: 'Terima Kasih!',
18 +       text: 'Anda telah berhasil berlangganan newsletter kami.',
19 +       icon: 'success',
20 +       background: isDarkMode ? '#111' : '#fff',
21 +       color: isDarkMode ? '#fff' : '#000',
22 +       confirmButtonColor: '#ffcc00',
23 +     });
24 +     setEmail('');
25 +   }
26 + }
```

▼ app  
+ page.tsx

▼ app/page.tsx

```
... @@ -5,6 +5,7 @@ import Link from 'next/link';
5 5 import Navbar from '@app/components/Navbar';
6 6 import Footer from '@app/components/Footer';
7 7 import WeatherWidget from '@app/components/WeatherWidget';
8 + import Newsletter from '@app/components/Newsletter';
8 9 import { useTheme } from '@app/context/ThemeContext';
9 10 import {
10 11   Compass, MapPin, Star, ArrowRight, ChevronLeft, ChevronRight,
... @@ -669,6 +669,9 @@ export default function PariwisataLampung() {
669 669 </div>
670 670 </section>
671 671 +
672 + { /* Newsletter Section */}
673 + <Newsletter />
674 +
672 675 { /* Footer component */}
673 676 <Footer />
674 677 </main>
```

▼ app  
+ page.tsx

▼ app/page.tsx

```
... @@ -669,6 +669,9 @@ export default function PariwisataLampung() {
669 669 </div>
670 670 </section>
671 671 +
672 + { /* Newsletter Section */}
673 + <Newsletter />
674 +
672 675 { /* Footer component */}
673 676 <Footer />
674 677 </main>
```

**Gambar 1 (Baris 51–67): Keterangan Pengarah Rute (*Router Context*)** Ditambahkan komentar penjelasan di samping baris kode `useRouter()`. Catatan ini memperjelas bahwa fungsi tersebut digunakan aplikasi untuk mengontrol navigasi atau perpindahan halaman (seperti mengarahkan kembali pengguna jika menekan tombol tertentu).

**Gambar 2 (Baris 42–58): Keterangan Status Tema (*Theme Context*)** Ditambahkan catatan di samping kode `useTheme()`. Catatan ini memperjelas bahwa baris tersebut berfungsi untuk membaca status sistem tema global yang sedang aktif (*dark mode* atau *light mode*) demi menyesuaikan tampilan visual halaman favorit.

**Gambar 3 (Baris 33–49): Dokumentasi Utama Komponen Halaman Favorit**  
Ditambahkan blok komentar deskripsi utama (JSDoc) di bagian atas fungsi komponen `FavoritPage`. Komentar ini menerangkan tanggung jawab utama dari halaman ini, yaitu untuk menampilkan seluruh daftar destinasi wisata yang telah ditandai sebagai favorit oleh pengguna.

**Gambar 4 (Baris 24–40): Batas Penutup Impor Komponen** Gambar ini menunjukkan batas akhir dari tumpukan kode impor modul dan bersiap masuk ke baris fungsi utama. Bagian hijau menandakan dimulainya penyisipan dokumentasi komponen yang rapi di bawah area impor.

**Gambar 5 (Baris 14–30): Pembersihan Teks Impor Tema** Menunjukkan penataan jarak baris (*spacing*) yang lebih rapi pada area atas berkas tempat mengimpor fungsi-fungsi bawaan, termasuk fungsi pengatur tema global aplikasi.

**Gambar 6 (Baris 108–124): Keterangan Penanganan Klik Kartu Wisata** Ditambahkan blok komentar dokumentasi di atas fungsi `handleCardClick`. Catatan ini menjelaskan fungsi dari kode di bawahnya, yaitu menangani aksi ketika salah satu kartu destinasi favorit diklik oleh pengguna, yang mana sistem akan mengarahkan mereka ke halaman detail dari tempat wisata tersebut.

**Gambar 7 (Baris 99–115): Keterangan Pengawas Sinkronisasi Data** Ditambahkan catatan penjelas di dalam fungsi efek (*useEffect*). Catatan ini menerangkan bahwa fungsi tersebut bertugas untuk mendaftarkan sistem pengawas (*event listener*) secara global. Tujuannya agar jika ada perubahan data favorit di halaman lain, daftar di halaman favorit ini langsung ikut diperbarui secara otomatis.

**Gambar 8 (Baris 86–102): Penutupan Fungsi Pembuat Efek** Gambar ini menunjukkan batas penutupan dari logika sinkronisasi data favorit. Komentar yang disisipkan memperjelas akhir dari proses pembacaan data awal dari memori penyimpanan lokal browser (*localStorage*) sesaat setelah halaman favorit dibuka.

**Gambar 9 (Baris 74–90): Keterangan Pengambil Data Awal Favorit** Ditambahkan komentar penjelasan tepat di dalam blok fungsi efek pertama. Catatan ini menegaskan fungsi baris kodenya, yaitu mengecek dan menarik data daftar destinasi favorit yang tersimpan di dalam memori lokal browser pengguna untuk kemudian dimasukkan ke dalam status aplikasi.

**Gambar 10 (Baris 61–77): Keterangan Pelacak Status Daftar Favorit** Ditambahkan komentar penjelasan di samping baris kode pembuatan status (*state*) favorites. Catatan baru ini menerangkan bahwa baris tersebut bertanggung jawab untuk menyimpan dan memantau susunan data (*array*) dari objek-objek destinasi wisata yang sedang difavoritkan oleh pengguna saat ini.

**Gambar 11 (Baris 190–206): Penutupan Grid Daftar Favorit** Gambar ini menunjukkan batas akhir penutupan komponen pembungkus tata letak grid. Ditambahkan baris kosong atau penataan lekukan kode (*indentation*) agar struktur pembungkus elemen daftar kartu favorit ini menutup dengan rapi tanpa merusak struktur halaman di bawahnya.

**Gambar 12 (Baris 183–199): Pengaturan Gaya Visual Kartu Wisata** Menunjukkan penataan ulang atau pembersihan spasi kode pada komponen kartu destinasi (DestinationCard). Kode yang ditandai warna hijau memastikan bahwa setiap kartu wisata favorit yang dipanggil lewat perulangan (*mapping*) menerima data properti yang pas dan disusun secara estetik.

**Gambar 13 (Baris 171–187): Batas Awal Perulangan Daftar Favorit** Gambar ini memperlihatkan awal mula dari fungsi perulangan untuk menampilkan daftar destinasi. Pengembang menambahkan baris rapi untuk memastikan jika pengguna memiliki daftar destinasi favorit, sistem akan langsung memetakan data tersebut ke dalam susunan baris kartu wisata.

**Gambar 14 (Baris 160–176): Keterangan Pembungkus Kisi (*Grid Layout*)** Ditambahkan komentar penjelas di atas tag HTML/JSX bagian tata letak. Catatan ini menerangkan bahwa area tersebut adalah kontainer utama berbentuk grid yang bertugas menyusun kartu-kartu destinasi wisata favorit agar tampilannya responsif (otomatis menyesuaikan jumlah kolom di layar HP maupun komputer).

**Gambar 15 (Baris 150–166): Keterangan Penanganan Jika Daftar Kosong** Ditambahkan catatan penjelasan di atas logika pengkondisian antarmuka. Komentar ini memperjelas alur kerja sistem: jika data favorit kosong (pengguna belum memilih destinasi apa pun), maka aplikasi akan menampilkan pesan atau ilustrasi ramah yang memberitahu bahwa halaman favorit masih kosong.

**Gambar 16 (Baris 140–156): Batas Penutup Area Header Halaman** Gambar ini menunjukkan bagian akhir dari dekorasi teks judul halaman "Destinasi Favorit Saya". Kode baru (warna hijau) merapikan tag penutup elemen pembungkus teks agar jarak ruang kosong (*padding/margin*) ke area daftar di bawahnya terlihat ideal.

**Gambar 17 (Baris 129–145): Keterangan Tombol Navigasi Kembali** Ditambahkan komentar penjelasan tepat di atas elemen tombol kembali (*Back Button*). Catatan ini menerangkan fungsi dari tombol tersebut, yaitu memicu fungsi navigasi untuk mengarahkan pengguna kembali ke halaman utama atau halaman jelajah sebelumnya saat diklik.

**Gambar 18 (Baris 121–137): Struktur Pembungkus Kontainer Utama Halaman** Menunjukkan peletakan tag pembungkus visual paling luar (*div*) untuk layout halaman

favorit. Tag ini diatur agar warna latar belakang, batas jarak tepi (*padding*), dan transisinya sinkron dengan tema gelap atau terang yang sedang aktif.

**Gambar 19 (Baris 112–128): Keterangan Aksi Berpindah ke Halaman Detail**

Ditambahkan catatan penjelasan di samping baris kode navigasi rute (`router.push`). Catatan ini mempertegas bahwa baris tersebut bertugas melempar pengguna ke alamat rute detail destinasi yang spesifik sesuai dengan ID kartu wisata yang mereka klik.

**Gambar 20 (Baris 108–124): Penutupan Blok Logika Klik Kartu** Gambar terakhir ini menunjukkan penutupan kurung kurawal fungsi penanganan klik kartu destinasi. Penataan ini memastikan alur perpindahan halaman dari daftar favorit menuju informasi lengkap tempat wisata tersebut selesai diprogram dengan bersih dan bebas dari eror sintaks.

**Gambar 21 (Baris 82–98): Keterangan Pengarah Rute (*Router Navigation*)** Ditambahkan komentar penjelasan di samping fungsi `useRouter()`. Catatan ini menerangkan bahwa fungsi navigasi dipersiapkan untuk memandu pengguna beralih halaman, contohnya ketika mereka menekan tombol kembali ke daftar favorit.

**Gambar 22 (Baris 72–88): Keterangan Pembacaan Parameter URL (*Search Params*)** Ditambahkan catatan di samping kode `useSearchParams()`. Catatan ini menjelaskan bahwa baris tersebut berfungsi untuk membaca data parameter yang ada di alamat URL browser (seperti mengambil data ID destinasi) agar sistem tahu detail tempat wisata mana yang harus dimuat.

**Gambar 23 (Baris 63–79): Keterangan Tema Global (*Theme Context*)** Ditambahkan komentar penjelas di samping fungsi `useTheme()`. Catatan ini menegaskan bahwa baris tersebut bertugas mengambil status tema yang sedang aktif (*dark/light*) guna menyesuaikan warna latar belakang halaman detail.

**Gambar 24 (Baris 54–70): Dokumentasi Utama Halaman Detail** Ditambahkan blok komentar JSDoc berukuran besar tepat di atas fungsi utama komponen `DetailPage`. Komentar ini menjelaskan tanggung jawab halaman, yaitu mengambil ID destinasi dari URL, memuat data lengkap tempat wisata tersebut, serta menampilkannya ke layar pengguna.

**Gambar 25 (Baris 45–61): Batas Atas Fungsi Utama** Gambar ini memperlihatkan penataan awal baris kode di dalam fungsi halaman detail. Blok hijau menandakan tempat dimulainya deklarasi status (*state*) dan pembacaan data rute sesaat setelah dokumentasi utama komponen dipasang.

**Gambar 26 (Baris 33–49): Merapikan Struktur Impor Modul** Menunjukkan penataan spasi dan baris kosong yang lebih bersih di bawah tumpukan kode impor. Hal ini dilakukan agar batas antara area impor berkas luar dan area penulisan logika halaman utama terlihat terpisah dengan jelas.

**Gambar 27 (Baris 24–40): Batas Penutup Daftar Impor** Gambar ini menampilkan pembersihan baris kode impor komponen UI pendukung. Penataan ini memastikan semua pustaka eksternal maupun lokal telah dipanggil dengan rapi sebelum masuk ke fungsi utama.

**Gambar 28 (Baris 14–30): Impor Komponen Antarmuka (*UI Components*)** Menunjukkan penataan ulang posisi baris impor untuk komponen visual seperti tombol kembali (*BackButton*) dan komponen kartu/detail lainnya yang akan dipakai untuk merancang visual halaman detail destinasi.

**Gambar 29 (Baris 112–128): Keterangan Logika Pemuatan Data Destinasi** Ditambahkan komentar penjelasan di dalam fungsi efek (*useEffect*). Catatan ini menerangkan alur pencarian data: sistem akan mengambil ID dari URL, lalu mencari data yang cocok dari daftar destinasi wisata yang tersedia, kemudian menyimpannya ke dalam status lokal aplikasi.

**Gambar 30 (Baris 102–118): Keterangan Pelacak Status Data Wisata** Gambar terakhir ini menunjukkan penambahan catatan di samping status destination. Catatan baru ini menegaskan bahwa status tersebut bertugas untuk menyimpan satu objek data destinasi pilihan pengguna secara utuh agar bisa dipecah nilainya (seperti nama tempat, foto, dan deskripsi) ke dalam elemen visual halaman.

**Gambar 31 (Baris 204–220): Batas Penutup Kontainer Teks Informasi** Gambar ini menunjukkan penataan tag penutup elemen teks. Langkah ini memastikan bahwa seluruh teks informasi mulai dari nama wilayah, ikon lokasi, hingga paragraf deskripsi tempat wisata terbungkus dengan rapi di dalam satu kotak tata letak yang sama.

**Gambar 32 (Baris 195–211): Penataan Paragraf Deskripsi Wisata** Menunjukkan pembersihan spasi dan penataan baris baru pada elemen paragraf yang menampilkan deskripsi lengkap tempat wisata (*destination.description*). Teks ini diatur agar memiliki warna dan ukuran yang nyaman dibaca, serta otomatis menyesuaikan ketebalannya berdasarkan tema aktif.

**Gambar 33 (Baris 186–202): Tampilan Wilayah Destinasi (*Location Tag*)** Ditambahkan elemen visual untuk menampilkan lokasi wilayah destinasi (*destination.location*). Bagian ini menggunakan ikon penunjuk peta kecil (*MapPin*) di samping teks nama wilayah untuk memberikan penanda visual geografis yang jelas bagi pengguna.

**Gambar 34 (Baris 174–190): Judul Nama Tempat Wisata** Menunjukkan penataan elemen teks judul utama halaman yang menampilkan nama dari destinasi wisata tersebut (*destination.name*). Ukuran huruf dibuat besar dan tegas agar langsung menjadi pusat perhatian pengguna saat halaman detail dimuat.

**Gambar 35 (Baris 165–181): Pembungkus Teks Informasi di Samping Foto** Gambar ini menunjukkan pembuatan kotak kontainer baru (*div*) khusus untuk menampung seluruh teks informasi wisata. Kotak ini diatur menggunakan sistem tata letak vertikal (*flex-col*) agar susunan judul, lokasi, dan deskripsi berjejer rapi ke bawah.

**Gambar 36 (Baris 156–172): Komponen Penampil Foto Utama Destinasi** Ditambahkan komponen gambar pintar (*Image*) bawaan Next.js untuk menampilkan foto utama dari destinasi wisata favorit tersebut. Komponen ini dilengkapi dengan sistem pengaman visual berupa efek sudut tumpul (*rounded-2xl*), bayangan lembut, serta optimasi ukuran agar gambar tidak pecah.



**Gambar 37 (Baris 144–160): Struktur Tata Letak Dua Kolom (*Grid Layout*)**

Menunjukkan pembuatan kontainer utama halaman menggunakan struktur grid dua kolom (`grid-cols-1 lg:grid-cols-2`). Aturan ini membuat halaman menjadi responsif: jika dibuka di HP, foto dan teks deskripsi akan menumpuk atas-bawah; namun jika dibuka di layar komputer, foto akan berada di sebelah kiri dan informasi teks di sebelah kanan.

**Gambar 38 (Baris 135–151): Penyisipan Tombol Navigasi Kembali** Ditambahkan penempatan fisik komponen tombol kembali (`BackButton`) di area paling atas halaman detail. Tombol ini diposisikan dengan jarak yang pas agar pengguna mudah mengekliknya kapan saja untuk kembali ke halaman daftar favorit sebelumnya.

**Gambar 39 (Baris 123–139): Penanganan Tampilan Jika Data Tidak Ditemukan**

Menunjukkan logika pengkondisian antarmuka (*conditional rendering*). Jika sistem selesai mencari data namun ternyata ID destinasi tidak ditemukan atau kosong, aplikasi tidak akan mogok (*crash*), melainkan otomatis menampilkan pesan error ramah di tengah layar bertuliskan "Destinasi tidak ditemukan".

**Gambar 40 (Baris 114–130): Batas Penutup Logika Pengecekan Awal** Gambar terakhir ini memperlihatkan batas penutup dari fungsi efek pemuatan data. Baris ini memastikan status pencarian data dari daftar destinasi selesai diproses dengan benar sebelum sistem memutuskan apakah akan menampilkan halaman detail utama atau memunculkan pesan error.

**Gambar 41 (Baris 38–54): Dokumentasi Fungsi Komponen Utama** Ditambahkan blok komentar JSDoc berukuran besar tepat di atas fungsi utama komponen `DestinasiPage`. Komentar ini menjelaskan tanggung jawab halaman tersebut, yaitu menampilkan daftar destinasi wisata, menangani fitur pencarian, serta melacak riwayat pencarian terakhir pengguna.

**Gambar 42 (Baris 29–45): Penutupan Definisi Struktur Data Destinasi** Gambar ini menunjukkan batas akhir penutupan struktur data (*interface*) dari objek Destinasi. Penataan ini memastikan semua properti pelengkap objek selesai didefinisikan dengan aman sebelum sistem masuk ke fungsi halaman utama.

**Gambar 43 (Baris 20–36): Definisi Properti Gambar dan Lokasi Destinasi** Ditambahkan kelanjutan properti ke dalam struktur tipe data objek destinasi. Baris hijau memastikan properti seperti teks `image` (untuk menyimpan alamat tautan foto) dan teks `location` (untuk menyimpan nama wilayah) terdaftar secara resmi sebagai tipe data teks (*string*).

**Gambar 44 (Baris 11–27): Definisi Properti Nama dan Deskripsi** Menunjukkan penambahan properti inti ke dalam struktur data objek destinasi. Bagian ini mendaftarkan properti nama destinasi (`name`) dan deskripsi lengkap tempat wisata (`description`) agar sistem dapat mengenali isi data dengan tepat saat dilakukan pencarian teks.

**Gambar 45 (Baris 7–23): Inisialisasi Struktur Data Objek Destinasi** Ditambahkan pembuatan struktur cetak biru data baru bernama `interface Destination`. Bagian awal ini mendaftarkan properti `id` dengan tipe data angka (*number*) sebagai tanda pengenal unik bagi setiap tempat wisata yang ada di dalam aplikasi.

**Gambar 46 (Baris 248–264): Pengaman Logika Pencarian Kosong** Menunjukkan kode pengaman visual di dalam komponen input teks. Baris baru yang disisipkan memastikan bahwa jika pengguna tidak mengetikkan apa pun atau menghapus seluruh teks pencarian, sistem akan langsung membersihkan tampilan antarmuka dan tidak memicu pencarian kosong yang sia-sia.

**Gambar 47 (Baris 238–254): Sinkronisasi Nilai Input dengan Riwayat** Ditambahkan baris pemacu agar ketika pengguna menekan tombol Enter pada sebuah kata kunci, sistem tidak hanya menyimpan kata tersebut ke memori browser, tetapi juga langsung menyinkronkan teks yang ada di dalam kotak input agar status pencarian langsung berjalan seketika.

**Gambar 48 (Baris 228–244): Pemasangan Logika Penyimpanan pada Tombol Enter** Gambar ini memperlihatkan pemanggilan fungsi `saveSearch(searchQuery)` secara resmi di dalam blok pendeteksi tombol Enter. Ini adalah jembatan yang menghubungkan aksi fisik kibor pengguna dengan fungsi otomatisasi memori yang membatasi riwayat maksimal 5 kata kunci tadi.

**Gambar 49 (Baris 218–234): Pengarah Kondisi Fokus Kolom Input** Menunjukkan penyelarasan tanda kurung dan pembatas logika pemacu tombol kibor. Kode baru ini memastikan alur perpindahan fokus layar (saat pengguna menekan tombol Enter) berjalan mulus dan kotak riwayat pencarian langsung menutup secara otomatis di momen yang tepat.

**Gambar 50 (Baris 208–224): Batas Penutup Blok Fungsi Interaksi** Gambar terakhir ini menunjukkan penataan baris penutup dari rangkaian logika interaksi kolom input pencarian. Langkah ini memastikan seluruh fungsi pengawas, mulai dari deteksi ketukan tombol Enter hingga fungsi simpan otomatis, terbungkus rapi tanpa ada masalah kebocoran kode (*syntax error*).

**Gambar 51 (Baris 29–45): Batas Akhir Penutup Komponen** Gambar ini menunjukkan bagian paling bawah dari berkas komponen. Kode baru yang ditandai warna hijau merapikan tanda kurung penutup fungsi komponen `ThemeSwitcher` serta memastikan perintah ekspor komponen (`export default`) berada di posisi yang tepat dan bersih.

**Gambar 52 (Baris 20–36): Keterangan Penyesuaian Ikon Tema** Ditambahkan komentar penjelasan di dalam elemen visual tombol (JSX). Catatan ini menerangkan bahwa sistem akan secara dinamis menampilkan ikon matahari (*Sun*) jika mode yang aktif adalah mode terang, dan otomatis berubah menjadi ikon bulan (*Moon*) jika pengguna beralih ke mode gelap.

**Gambar 53 (Baris 11–27): Keterangan Fungsi Tombol Klik (*Toggle Theme*)** Ditambahkan komentar penjelasan tepat di samping fungsi pembuka tombol klik. Catatan ini mempertegas alur kerja tombol: saat diklik oleh pengguna, tombol ini akan langsung memicu fungsi sakelar untuk membalikkan status tema global saat ini (dari terang ke gelap, atau sebaliknya).

**Gambar 54 (Baris 7–23): Keterangan Pembacaan Status Tema Global** Ditambahkan komentar penjelasan di samping baris kode pengait tema (`useTheme`). Catatan baru ini

menerangkan bahwa baris tersebut berfungsi untuk mengambil data tema aktif beserta fungsi pengubahnya dari pusat pengaturan aplikasi (*Theme Context*).

**Gambar 55 (Baris 1–17): Dokumentasi Utama Komponen Sakelar Tema** Ditambahkan blok komentar JSDoc berukuran besar di bagian paling atas berkas (sebelum fungsi dimulai). Komentar ini menjelaskan tanggung jawab utama dari komponen ThemeSwitcher, yaitu menyediakan tombol interaktif bagi pengguna untuk mengubah skema warna aplikasi antara mode terang (*light*) dan mode gelap (*dark*).

**Gambar 56 (Baris 108–124): Penutupan Data Objek Destinasi Terakhir** Gambar ini menunjukkan bagian akhir dari daftar susunan (*array*) data wisata. Penataan ini memastikan tanda kurung siku penutup daftar data tersusun dengan rapi agar tidak menyebabkan error pembacaan data pada sistem.

**Gambar 57 (Baris 99–115): Dokumentasi Data Wisata Kelima (Raja Ampat)** Menunjukkan penataan susunan properti objek wisata kelima, yaitu destinasi **Raja Ampat**. Seluruh data mulai dari ID, nama, deskripsi keindahan bawah laut, alamat foto, hingga lokasi wilayahnya dipastikan tertulis dalam format terstruktur yang bersih.

**Gambar 58 (Baris 86–102): Dokumentasi Data Wisata Keempat (Gunung Bromo)** Menunjukkan penataan susunan properti objek wisata keempat, yaitu destinasi **Gunung Bromo**. Data ini memuat informasi lengkap mengenai keindahan pemandangan matahari terbit di Bromo untuk dikonsumsi oleh halaman destinasi dan favorit.

**Gambar 59 (Baris 74–90): Dokumentasi Data Wisata Ketiga (Tana Toraja)** Menunjukkan penataan susunan properti objek wisata ketiga, yaitu destinasi **Tana Toraja**. Struktur data ini menyimpan informasi warisan budaya dan rumah adat tradisional Toraja secara lengkap.

**Gambar 60 (Baris 61–77): Pembatas Antar Objek Data Wisata** Gambar terakhir ini memperlihatkan tanda koma pembatas dan penataan ruang kosong (*spacing*) yang memisahkan antara objek data wisata kedua (seperti Candi Borobudur) dengan objek data wisata ketiga di dalam daftar memori aplikasi.

**Gambar 61 (Baris 48–64): Batas Penutup Data Objek Wisata Kedua** Gambar ini menunjukkan penataan tanda kurung kurawal penutup untuk data objek wisata kedua (**Candi Borobudur**). Pengembang merapikan baris kosong di bawah properti lokasi (*location*) agar baris penutup ini memisahkan data Borobudur dengan data destinasi berikutnya secara terstruktur.

**Gambar 62 (Baris 38–54): Properti Gambar dan Lokasi Candi Borobudur** Menunjukkan pembersihan spasi pada properti pelengkap objek wisata kedua. Di sini dipastikan penulisan properti alamat gambar (*image*) dan nama wilayah (*location*: "Magelang, Jawa Tengah") tidak memiliki spasi berlebih di awal baris (*indentation*).

**Gambar 63 (Baris 29–45): Properti Nama dan Deskripsi Candi Borobudur** Menunjukkan penataan teks informasi utama untuk objek wisata kedua. Properti *name*: "Candi Borobudur" dan paragraf deskripsi (*description*) mengenai kemegahan candi Buddha terbesar di dunia ini dirapikan posisinya agar sejajar dengan properti lainnya.

**Gambar 64 (Baris 20–36): Batas Pemisah Antara Objek Bali dan Borobudur** Gambar ini memperlihatkan tanda koma pembatas yang memisahkan objek pertama (**Pulau Bali**) dengan objek kedua (**Candi Borobudur**). Sekaligus menunjukkan inisialisasi awal data Borobudur yang diberi tanda pengenalan angka unik, yaitu id: 2.

**Gambar 65 (Baris 11–27): Penataan Properti Gambar dan Lokasi Pulau Bali** Gambar terakhir ini menunjukkan perbaikan posisi baris teks pada bagian akhir objek wisata pertama, yaitu **Pulau Bali**. Properti tautan gambar (image) dan lokasi (location: "Bali, Indonesia") digeser sedikit agar sejajar secara vertikal dengan properti nama dan deskripsi di atasnya, membuat keseluruhan berkas data ini menjadi sangat bersih dan mudah dibaca oleh sistem aplikasi.